HOMEMESH PANEL

A Wall-Mounted Smart Home Control Panel using ESP32 and the Home Assistant WebSocket API

CSE3006 — Embedded System Design

Final Review — Project Documentation

Project title	HomeMesh Panel — local-first smart home control panel
Institution	Vellore Institute of Technology, Chennai
Course	CSE3006 — Embedded System Design
Team members	Garima Nidhi Taniska Routray Bharadhwaj B Pranesh S
Register numbers	24BCE1782 (Garima Nidhi) 24BCE1578 (Taniska Routray) 24BCE1959 (Bharadhwaj B) 24BCE5138 (Pranesh S)
Faculty guide	Suhasini S
Date of submission	21 October 2026
Repository	github.com/agent192007/homemesh-panel (private)

This version: 20 October 2026. It replaces the version of 17 September, which described the project before the layout moved onto the network and before the desktop designer existed.

Table of Contents

How to Use This Document 4

1. Abstract 5

2. Introduction 5

2.1 Home Assistant 5

2.2 Literature survey 6

3. Problem Statement 6

4. Objectives 7

5. Existing Systems and Their Limitations 7

5.1 Why the MQTT approach was rejected 7

5.2 The three designs the project went through 8

6. Proposed System 8

6.1 The memory constraint and the split architecture 8

6.2 The layout is data, not code 9

7. System Architecture 10

7.1 Overall system diagram 10

7.2 Hardware diagram 11

7.3 Software diagram 11

7.4 Block diagram 12

8. Hardware Description 13

8.1 Components 13

8.2 Pin assignment 13

8.3 Batch variation in this board family 13

9. Software Description 15

9.1 Panel firmware: environment and libraries 15

9.2 Desktop designer: environment and libraries 15

9.3 Source code organisation 15

9.4 The transport interface 16

10. Implementation 17

10.1 The communication protocol 17

10.2 Task structure and interface responsiveness 18

10.3 Memory strategy 18

10.4 Touch coordinate handling 18

10.5 Receiving and storing a layout 18

10.6 The user interface 19

10.7 Themes 20
10.8 A panel that explains its own failures 21
11. The HomeMesh Designer (Desktop App) 22
11.1 What it does 22
11.2 The simulator is the real interface 22
11.3 How the designer is tested 23
12. System Operation 24
12.1 Network configuration 24
12.2 Devices on the panel 24
12.3 Changing what the panel shows 24
13. Testing and Results 25
13.1 Message size and latency 25
13.2 Resource usage 25
13.3 Functional verification 26
13.4 Field case study: the connection that kept dropping 26
13.5 Other challenges and their resolution 27
14. Future Scope 27
15. Conclusion 27
16. References 28
16.1 Published literature 28
16.2 Technical documentation 29
Appendix A — Suggested Presentation Plan 30
Figure files 30
Presentation notes 31
Anticipated questions 31
Appendix B — Internal Note (Not for Presentation) 33
What is genuinely working 33
What must not be claimed 33
Check before the review: firmware on the board 34
Two quirks that may be noticed, and how to answer 34
Before the presentation 34

How to Use This Document

This document is written for a team member who has not worked on the code, and who is building the presentation from it. It assumes no prior knowledge of the project and explains each term the first time it appears.

Sections 1 to 12 follow the structure the course expects and map almost one-to-one onto slides. Section 13 has the measured results and the problems solved along the way. **Appendix A** is a slide-by-slide plan with the figure to use on each slide and likely questions with their answers. **Appendix B** is an internal note for the team only: what can be claimed, what cannot, and the checklist for the morning of the review.

Every figure in this document is also a separate image file, listed in Appendix A, and can be dropped straight onto a slide. Every number was measured on the running system, not estimated.

Read Appendix B before presenting

Appendix B lists what is not finished and the specific claims that must not be made during the review. Everything else in this document is safe to present.

What changed since the 17 September version

- The device list is **no longer compiled into the firmware**. It is designed on a laptop and sent to the panel over the network, where it is stored. No cable, no reflashing. Proven on the board on 17 October.
- A new **desktop designer app** replaces the command-line tool. Its preview is the panel's own interface code running on the PC, not a drawing of it.
- The panel now has **device icons**, a **new tile design**, **custom device names**, and **six colour themes**, all chosen in the designer.
- The panel takes device state over a **single subscription** (`subscribe_entities`) instead of one request per device. This fixed a connection-loss bug, described in Section 13.4.
- A **Wi-Fi fault** that looked like random firmware failure was traced to the laptop's Wi-Fi card. It makes a good story for the review, also in Section 13.4.

1. Abstract

HomeMesh Panel is a wall-mounted touchscreen that controls the devices in a home automation system. It is built on an ESP32 microcontroller with a 3.2-inch capacitive touchscreen, and it communicates with a Home Assistant server over the local wireless network using the Home Assistant WebSocket API.

The panel shows lights, switches, fans and environmental sensors as touchable tiles, on pages that correspond to rooms, plus a Home page of favourites. A tap switches a device; a detail panel sets brightness and colour. State travels in both directions: a change made anywhere else in the system appears on the panel within milliseconds, because the server pushes every change to it.

The central engineering problem is memory. Asking Home Assistant for the state of every device returned **57,671 bytes for the 123 devices** of the test installation, more than the panel's entire 40 KB graphics memory, on a chip with no external RAM. The project solves this by splitting the work across two machines: a desktop application performs discovery and designs the panel's layout, and the panel subscribes only to the devices in that layout. Its initial state for seven devices arrives in a single **2,403-byte** message.

The layout — which devices, on which pages, with which names, icons and colours — is data, not code. It is sent to the panel through Home Assistant and stored in the panel's non-volatile memory, so the panel is reconfigured in seconds without reprogramming. The desktop designer previews the result using the panel's own interface code compiled to run on a PC.

The system runs entirely on the local network. It needs no cloud service, no vendor app and no internet connection.

2. Introduction

Home automation systems have become common, but the way people interact with them has not kept pace. Most are controlled through a smartphone app, so switching on a light means unlocking a phone, finding an app, waiting for it to connect and navigating to the right screen. That is slower and less reliable than the wall switch it replaced, and it leaves out visitors and anyone without the phone.

A wall-mounted control panel removes that friction. It is always on, always connected and always in the same place. It behaves like a light switch, but it can control any device in the system and show information a switch cannot, such as room temperature and humidity.

Commercial wall panels exist, but they are expensive, usually tied to one manufacturer, and often depend on a cloud service. A cloud dependency means a device two metres away stops responding when the internet connection fails, and that a record of household activity leaves the home.

This project builds such a panel from inexpensive, widely available parts, controls an open-source home automation platform, and works entirely within the local network.

2.1 Home Assistant

Home Assistant is an open-source home automation platform: the software that actually talks to smart devices. It supports a very large number of device types through components called *integrations* — one for Zigbee devices, another for Z-Wave, another for Wi-Fi devices, and so on.

The important consequence is that Home Assistant presents all of these devices through one uniform interface, whatever the underlying wireless technology. Each controllable thing is called an *entity*, with an

identifier such as `light.bed_light`. The panel is a client of Home Assistant: it never talks to bulbs or switches directly, it asks Home Assistant to.

2.2 Literature survey

The project draws on four strands of published work. The full references are in Section 16.1.

Area	What the literature says	What HomeMesh takes from it
Edge and local processing [1], [2]	Processing inside the home, rather than in a vendor cloud, reduces latency, keeps working without the internet and keeps household data local.	No cloud anywhere in the path. Server, network and panel are all on the premises.
Interoperability [3], [4]	Device fragmentation across Zigbee, Z-Wave, Wi-Fi and vendor clouds has been the central problem of home automation for two decades; Matter is the latest attempt to solve it.	Talk to Home Assistant, which already integrates devices from many ecosystems, instead of to devices.
ESP32 home automation [5], [6], [7]	Typical ESP32 systems switch relays and read sensors wired to the board, and are controlled from a web page or cloud dashboard.	The ESP32 is the controller, not the controlled: one panel for every device the server knows.
Accessible fixed panels [8]	A fixed panel gives every occupant, including people without a smartphone, control of the room they are in.	The motivation for a wall-mounted panel at all.
IoT protocols [9]–[12]	MQTT is the dominant IoT messaging protocol; surveys compare it with CoAP, HTTP, AMQP and others.	Why the first version used MQTT, and the reach problem that made it a dead end (Section 5.1).
MQTT vs WebSocket [13]–[16]	WebSocket gives the lowest latency for interactive, request-driven exchanges; MQTT is more economical for streaming telemetry.	A control panel is interactive by nature, which favours WebSocket.
ESP32 power [17]	Measured power consumption of the ESP32 in low-power applications.	Not a goal here: the panel is mains powered, so Wi-Fi power saving is switched off for responsiveness.

3. Problem Statement

Design and build a wall-mounted touchscreen control panel for a home automation system that satisfies these conditions:

1. It must control real devices of different types through one interface, without being restricted to one wireless technology or one manufacturer.
2. It must show the true current state of each device, and update when that state is changed by any other means.
3. It must work entirely on the local network, with no cloud service in the control path.
4. It must run on a low-cost microcontroller with severely limited memory, which rules out the straightforward approach of downloading the entire system state.
5. It must be reconfigurable — which devices, where, how they look — by someone with no programming toolchain.
6. The interface must stay responsive to touch at all times, including while the network is busy.

4. Objectives

1. To interface an ESP32 microcontroller with a 320 × 240 colour display and a capacitive touch panel, and to identify the controller variants actually fitted to the board.
2. To implement a WebSocket client on the ESP32 that authenticates with Home Assistant and holds one persistent connection to it.
3. To receive device state pushed by the server and reflect it on the display without polling.
4. To send control commands from the panel so that devices can be switched, dimmed and coloured from the touchscreen.
5. To design a memory strategy that lets a device with no external RAM take part in a system whose full state is far larger than its memory.
6. To build a multi-page, room-based interface that stays responsive during network activity.
7. To make the panel's layout data rather than code: authored on a PC, delivered over the network, stored on the panel, with no reflashing.
8. To build a desktop designer that discovers devices from the server and previews the panel exactly as it will appear.

5. Existing Systems and Their Limitations

Three categories of existing solution were examined before the architecture was chosen.

Approach	How it works	Limitations
Manufacturer wall panels	A sealed product sold as part of one manufacturer's ecosystem.	High cost. Controls only that manufacturer's devices. Usually needs a cloud account, so it fails when the internet fails. Not extensible.
Tablet running a web dashboard	A consumer tablet on the wall showing the home automation web interface in a browser.	Expensive for what it does. Slow to boot. The battery degrades on permanent charge. A full operating system to maintain and update.
Microcontroller panel using MQTT	An ESP32 connected to an MQTT message broker, publishing and subscribing to topics that represent devices.	Only reaches devices that publish themselves to the broker. Large classes of device are invisible. See Section 5.1.

5.1 Why the MQTT approach was rejected

MQTT is a lightweight publish-subscribe messaging protocol widely used in embedded systems, and it was the approach used in the first version of this project. It was abandoned, and the reason is the most important design decision in the work.

MQTT can only reach devices that publish themselves onto a message broker. A device connected to Home Assistant through a non-MQTT integration — a Philips Hue bulb, a Z-Wave switch, or any device added through the ZHA Zigbee integration — exists entirely inside Home Assistant and has no MQTT topic at all. A panel speaking MQTT cannot see such a device, and so cannot control it.

This is not just a limitation; it is unpredictable for the user. **The same physical Zigbee bulb** is reachable over MQTT if it was added through Zigbee2MQTT, and completely invisible if it was added through ZHA. The hardware is identical. Whether the panel can control it depends on a setup decision the user had no

reason to think about. A panel whose coverage depends on how each device happened to be added is not a usable product.

5.2 The three designs the project went through

Design	Devices reached	How state arrived	Why it was replaced
1. MQTT broker (Sept)	Only those publishing to the broker	Retained MQTT messages	Most Home Assistant devices invisible
2. WebSocket + REST (Sept – 17 Oct)	Every Home Assistant entity	One HTTP request per device, then WebSocket events	The requests blocked the connection and caused reconnect loops (Section 13.4)
3. WebSocket only (18 Oct, final)	Every Home Assistant entity	One subscription: a snapshot, then only changes	—

6. Proposed System

The proposed system replaces MQTT with the Home Assistant WebSocket API — the same interface Home Assistant's own web interface uses. It has two properties MQTT cannot offer:

- **Uniform reach.** Every entity Home Assistant knows is available, whichever integration created it. A Zigbee bulb, a Z-Wave switch, a Wi-Fi plug and a software-only helper are all addressed the same way.
- **One control command.** A single message type, `call_service`, operates all of them. Switching a Zigbee bulb and switching a Wi-Fi plug are the same message with a different target.

A *WebSocket* is a network connection that stays open. With an ordinary web request the client asks, the server answers and the connection closes; a WebSocket stays established so either side can send at any time. That is what lets the server push a change to the panel the moment it happens, instead of the panel repeatedly asking (*polling*), which wastes power and network capacity and adds delay.

6.1 The memory constraint and the split architecture

Adopting the WebSocket API brings a problem. The natural first step for any client is to ask for the state of everything, with a request called `get_states`. On the test installation that returned **57,671 bytes for 123 entities** — about 469 bytes each. A real house with a few hundred devices would be hundreds of kilobytes.

The ESP32-WROOM-32 has no external RAM (PSRAM), and its graphics library works inside a fixed 40 KB pool. Even the small test installation's full state is bigger than that. This is not a problem that more careful code can solve; the data is simply larger than the memory.

So the work is split across two machines. **This split is the central design idea of the project:**

- **The PC does discovery.** The desktop designer connects to Home Assistant, calls `get_states`, shows every entity grouped by room, and the user picks the handful the panel should show.
- **The panel never does discovery.** It subscribes to exactly the entities in its layout and sends commands only to them. Every message it handles is small and bounded by construction.

Measured effect: the panel's initial state for its seven entities arrives in one **2,403-byte** message — 24 times smaller — and that size grows with the layout, not with the size of the house.

6.2 The layout is data, not code

In the first versions, the list of devices was compiled into the firmware: changing it meant rebuilding and reflashing over USB. It is now data. The designer produces a small JSON document (about 700 bytes) naming the rooms, the devices, their names, icons and kinds, the sensors, and optional theme colours. It is sent to the panel through Home Assistant and stored in the panel's non-volatile memory (NVS — flash memory that survives power loss). The firmware never changes; only the layout does.

```
{"v":1,
 "areas":["Bedroom","Kitchen","Living Room"],
 "entities":[
   {"id":"light.bed_light", "name":"Bed Light", "kind":"RGB",
    "area":0, "fav":true, "icon":"light"},
   ...],
 "sensors":[{"id":"sensor.outside_temperature", "area":2, "hum":false}, ...],
 "theme":{"bg":"#F2F3F7", ...}     <- optional: only colours that differ
}
```

7. System Architecture

The architecture is shown in three views, following the course convention: an overall system diagram of the major blocks and the data between them, a hardware diagram of the physical components and their electrical interfaces, and a software diagram of the program structure. Section 7.4 gives a combined block diagram used in the paper.

7.1 Overall system diagram

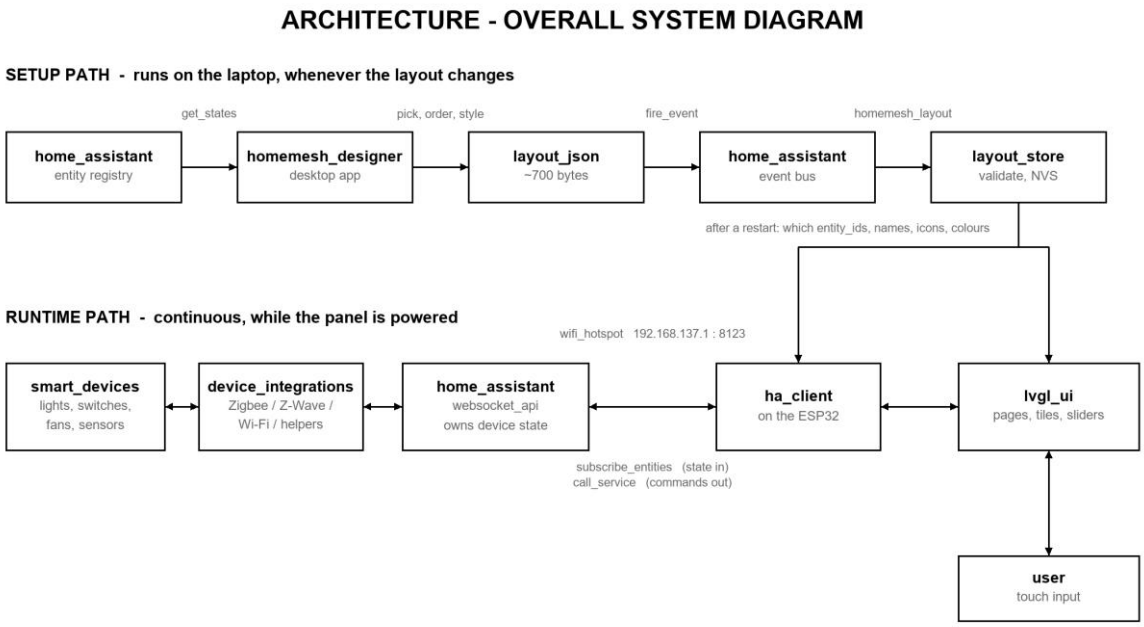


Figure 1. Overall system architecture. The setup path runs on the laptop whenever the layout changes; the runtime path runs continuously on the panel.

Setup path (top). The designer asks Home Assistant for every entity, the user chooses and arranges them, and the designer sends the resulting layout as a Home Assistant event called `homemesh_layout`. Home Assistant relays it to the panel over the connection the panel already holds. The panel validates it, stores it in NVS and restarts into it. No web server runs on the panel, and neither side ever needs the panel's IP address — both only need to know where Home Assistant is.

Runtime path (bottom). Physical devices are controlled by Home Assistant through their integrations. The panel holds one WebSocket connection to Home Assistant over the laptop's Wi-Fi hotspot, receives state through it and sends commands back through it. The user interacts by touch.

7.2 Hardware diagram

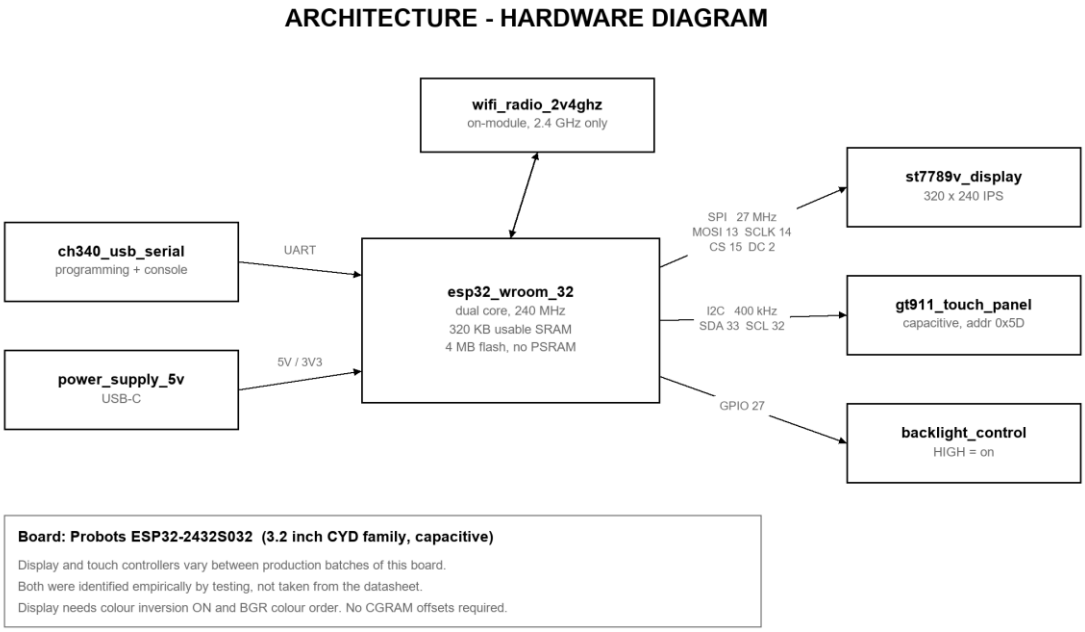


Figure 2. Hardware architecture: the microcontroller and its peripheral interfaces.

The ESP32-WROOM-32 module is the centre of the system. The display is driven over a four-wire SPI bus at 27 MHz; the touch panel is read over I²C at 400 kHz; the backlight is switched by one GPIO pin. A CH340 chip provides the USB-to-serial link used for programming and for the diagnostic log. The 2.4 GHz Wi-Fi radio is built into the module.

7.3 Software diagram

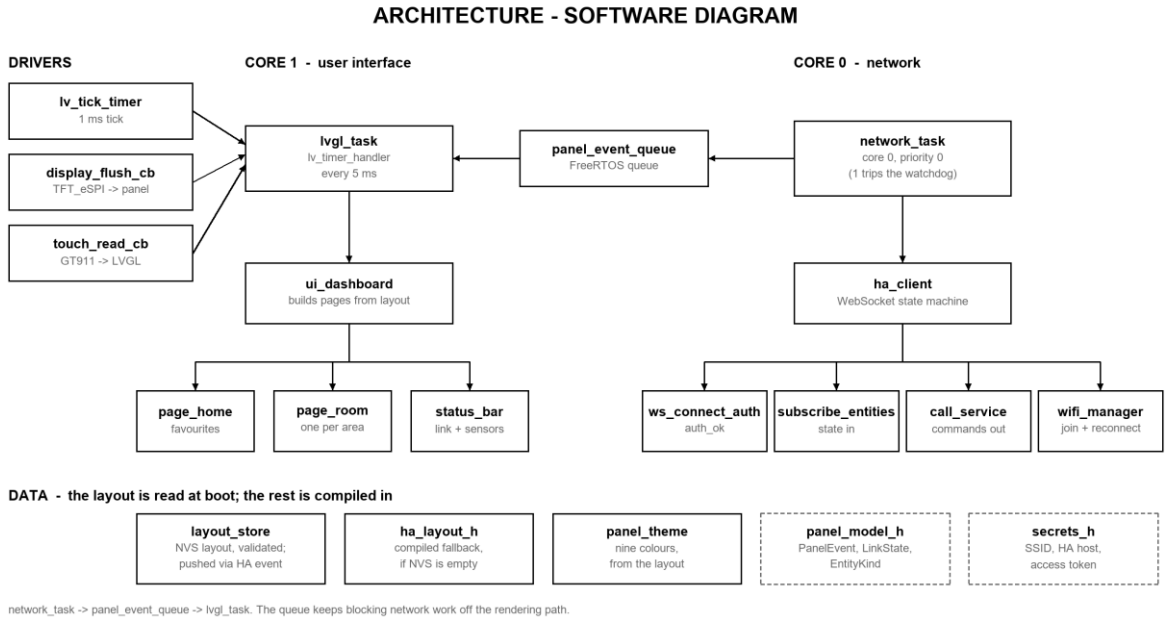


Figure 3. Software architecture. The two processor cores are separated by a queue; the layout is read from NVS at boot.

The software is split across the ESP32's two cores. Core 1 runs the user interface; core 0 runs the network client. They never call into each other; they communicate through a FreeRTOS queue — a thread-safe buffer one task writes and the other reads. This separation is why the interface stays smooth while the

network is busy (Section 10.2). The bottom row shows what drives the firmware: the layout from NVS (with a compiled-in fallback), the theme colours, and the compiled configuration.

7.4 Block diagram

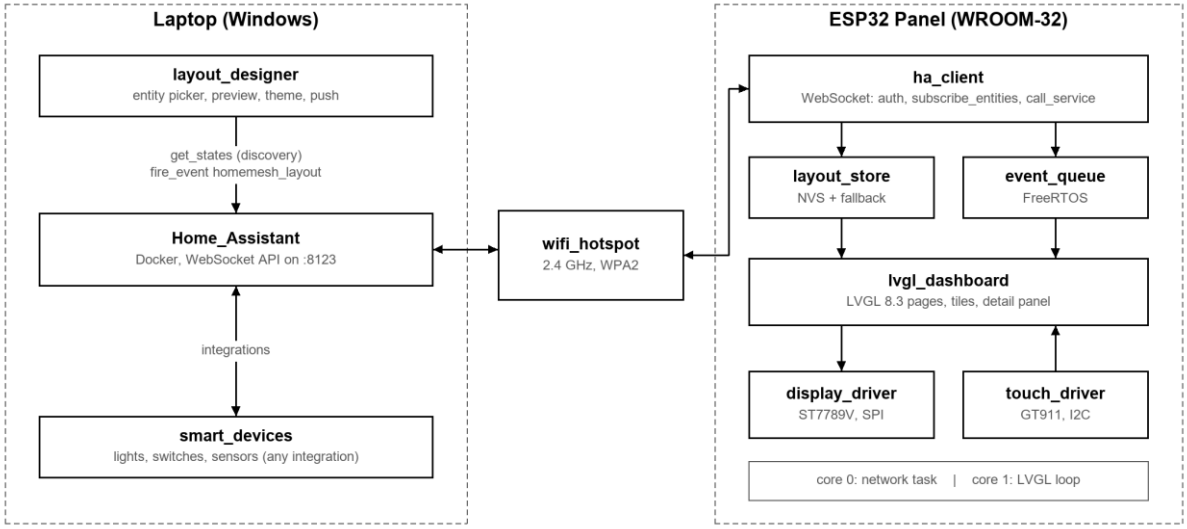


Figure 4. System block diagram, as used in the IEEE paper.

8. Hardware Description

The project uses a **Probots ESP32-2432S032** development board, from a family nicknamed the "Cheap Yellow Display", which puts an ESP32 module, a colour display and a touch panel on one board.

8.1 Components

Component	Part	Specification and role
Microcontroller	ESP32-WROOM-32	Dual-core 32-bit processor at 240 MHz. 520 KB on-chip SRAM, no external PSRAM. 4 MB flash. Integrated 2.4 GHz Wi-Fi (no 5 GHz). Runs the whole application.
Display	ST7789V controller 3.2" 240 × 320 IPS	Used in landscape at 320 × 240, 65,536 colours (16-bit RGB565). SPI at 27 MHz. Needs colour inversion on and BGR colour order.
Touch panel	Goodix GT911	Projected capacitive. I²C at 400 kHz, address 0x5D.
USB interface	CH340	USB-to-serial for firmware upload and the diagnostic log.
Storage	4 MB SPI flash	3 MB application partition, plus NVS where the pushed layout is stored.
microSD slot	128 GB card fitted	Physically present but **not used and not yet tested**. Candidate uses: logging, assets (Section 14).
Power	5 V over USB-C	On-board regulator supplies 3.3 V.

8.2 Pin assignment

These values are specific to this board. They are set as build flags in `platformio.ini`, not in the display library's own configuration file.

Signal	Pin	Interface	Function
MOSI	13	SPI	Serial data to the display
SCLK	14	SPI	Serial clock for the display
CS	15	SPI	Display chip select
DC	2	SPI	Data / command select
MISO	12	SPI	Serial data from the display
RST	−1	SPI	Not connected; reset handled internally
BL	27	GPIO	Backlight enable, active high
SDA	33	I²C	Touch panel data
SCL	32	I²C	Touch panel clock

8.3 Batch variation in this board family

Boards in this family are sold under the same name while carrying different display and touch controllers from batch to batch, so published tutorials are often wrong for any particular unit. The parts on this board were identified experimentally: an I²C scanning program found the touch controller and its address, and a

display test program confirmed the display controller by seeing which initialisation produced a correct image.

The result: an **ST7789V** display controller, not the ILI9341 most tutorials assume, and a **GT911** capacitive touch controller. These symptoms identify a colour fault immediately if one ever recurs:

Symptom	Cause
Red and blue swapped; green, white and black correct	Colour order wrong — the panel is BGR, not RGB
All colours inverted (red shows as cyan)	Display inversion setting wrong for this panel
Image shifted with a noise band along one edge	Memory offsets wrong. This board needs none, so it did not occur

Because of this variation, all hardware-specific code lives in two driver files. The interface and network code never see it — a decision that later made the transport rewrite cheap.

9. Software Description

9.1 Panel firmware: environment and libraries

Layer	Choice	Version	Reason
Build system	PlatformIO	—	Libraries, build configurations and upload. No definition exists for this board, so the generic esp32dev target is used.
Framework	Arduino for ESP32, on FreeRTOS	—	Widest library support; almost all reference material for this board assumes it.
Graphics	LVGL	8.3.11	Buttons, sliders, colour wheel, pages. Pinned to 8.3 deliberately: version 9 changed the driver interfaces and almost every example for this board is 8.3.
Display driver	TFT_eSPI	2.5.43	Fast SPI display driver with ST7789V support.
WebSocket	links2004 WebSockets	2.7.3	Maintained WebSocket client for the ESP32.
JSON	ArduinoJson	6.21.6	Filtered, fixed-size parsing with bounded memory.
Icons	Material Design Icons as LVGL fonts	16 glyphs	Drawn like text from flash: about 7.6 KB of flash and nothing from the graphics memory.
Partitions	huge_app.csv	—	3 MB application partition instead of 1.3 MB.

9.2 Desktop designer: environment and libraries

Layer	Choice	Reason
Application shell	Electron	A desktop app that runs on any Windows PC as a single portable .exe; nothing to install.
Interface	React 19 + TypeScript	Typed, component-based interface; the layout format is checked at compile time.
Build	Vite	Fast builds and hot reload during development.
Panel simulator	Emscripten (C/C++ → WebAssembly)	Compiles the panel's own interface code, LVGL and fonts to run inside the app. See Section 11.2.
Home Assistant client	Browser WebSocket	The same protocol the panel uses, without its memory limits.
Reference tools	Python 3: ha_discover.py, preflight.ps1	The layout format is defined once in Python; the app's version is tested against it byte for byte.

9.3 Source code organisation

Hardware details are confined to two files and never appear in the interface or network code.

File	Responsibility
src/main.cpp	Entry point. Starts the drivers and the network task, runs the graphics loop, and reports memory use and graphics failures.
src/display_driver.*	Display start-up and the callback that sends rendered pixels to the panel.

File	Responsibility
src/touch_driver.*	Touch controller and the callback that reports touches, including rotation.
src/ha_client.*	The Home Assistant WebSocket client: connect, authenticate, subscribe, send commands, receive layouts.
src/layout_store.*	The layout: read from NVS at boot, validate, store new ones, fall back to the compiled layout.
src/panel_theme.*	The nine theme colours, defaulted and overridden by the layout.
src/ui/dashboard.*	All screens: pages, tiles, the detail panel, the automations screen.
src/ui/panel_icons.*, fonts/	Icon name → glyph, and the icon fonts.
src/panel_model.h	The contract between transport and interface (Section 9.4).
src/mqtt_client.*, fake_client.cpp	The original MQTT transport and an offline simulated one. Unused in the demo, kept as fallbacks.
include/ha_layout.h / .json	The compiled-in fallback layout, used when nothing has been pushed.
include/secrets.h	Wi-Fi name and password, server address, access token. Never committed.
designer/	The desktop designer app (Section 11).
sim/	Builds the panel's interface to WebAssembly for the designer.
tools/	ha_discover.py (discovery and layout format), preflight.ps1 (demo-day checks), the older tkinter designer.
docs/	The IEEE paper, this document and the scripts that draw every figure.

9.4 The transport interface

`panel_model.h` is the boundary between the network code and the interface code. The network side knows nothing about buttons; the interface knows nothing about sockets. They share only three definitions:

Definition	Meaning
EntityKind	What controls a device gets. Toggle: on/off. Dimmable: adds brightness. RGB: adds colour.
LinkState	Connection condition — no Wi-Fi, no server, or online. Drives the status dot.
PanelEvent	A small fixed-size record of one change: which device, what changed, the new value. Copied through the queue.

The value of this boundary showed when MQTT was replaced by the WebSocket client: **this file did not change at all**. The new transport offered the same three functions as the old one — start, next event, send — so the main program and the interface barely changed. A potential rewrite was contained to one file.

10. Implementation

10.1 The communication protocol

From the panel's point of view, the whole of a home automation system reduces to five exchanges on one connection:

```
1.  Connect     ws://192.168.137.1:8123/api/websocket
                server:  {"type":"auth_required"}

2.  Log in      {"type":"auth","access_token":"<long-lived token>"}
                server:  {"type":"auth_ok"}

3.  Subscribe   {"id":1,"type":"subscribe_entities","entity_ids":[...]}
                server:  one snapshot of every listed device   {"event":{"a":{...}}}
                         then only what changes                {"event":{"c":{...}}}

4.  Listen      {"id":2,"type":"subscribe_events","event_type":"homemesh_layout"}
                server:  layouts sent from the designer

5.  Control     {"id":N,"type":"call_service","domain":"light","service":"turn_on",
                 "target":{"entity_id":"light.bed_light"},"service_data":{"brightness":180}}
```

Message 3 is how state comes in, message 5 is how commands go out, and message 4 is how the panel is reconfigured. There is nothing else and no second connection.

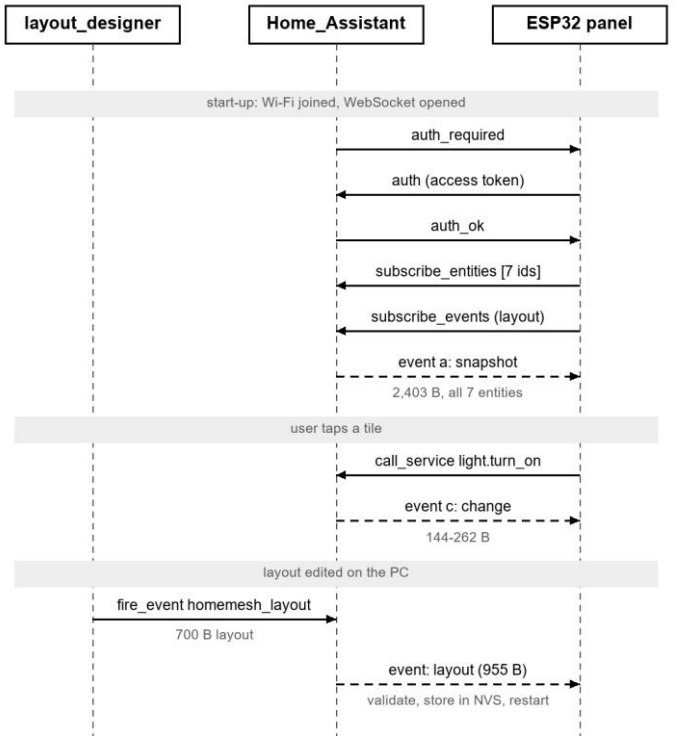


Figure 5. Message sequence between the panel and Home Assistant.

Availability is the connection itself. The old MQTT version used a broker feature called a Last Will and Testament for this; the WebSocket API needs none. If the socket drops, the panel greys its tiles out immediately, and a 15-second heartbeat catches a connection that dies silently.

10.2 Task structure and interface responsiveness

The graphics library must be serviced about every 5 ms to read touches, animate and redraw. Anything that blocks for longer causes visible stutter — and network operations routinely block for hundreds of milliseconds.

So the network client runs as a separate FreeRTOS task on core 0 and the interface on core 1. The network task never touches the interface: it turns each change into a `PanelEvent` and puts it on a queue, and the interface drains the queue on each pass of its loop. A network operation that blocks for half a second has no effect on the screen; pages still swipe and sliders still track the finger.

> **Detail worth knowing: task priority**
>
> The network task runs at FreeRTOS priority 0, not 1. At 1 it starves the idle task on core 0, which is responsible for feeding the hardware watchdog; the watchdog concludes the system has hung and restarts it. Symptom: a panel that reboots every few seconds for no visible reason.

10.3 Memory strategy

Memory shaped almost every implementation decision.

- **Program storage.** The default 1.3 MB application partition was 92% full. A 3 MB partition brought it to 38%. The cost is the over-the-air update slot, which this project does not use.
- **Graphics memory** is a fixed 40 KB pool. Running out does not give an error message; it freezes or blanks the screen. The panel logs its headroom at every boot (about 15 KB free).
- **Static memory is the real ceiling.** The build report says about 37% of RAM is used, but the linker's data segment is the true limit, and the firmware sits about 1.5 KB below it: a 48 KB graphics pool, or even one extra 2 KB buffer, failed to link. Large buffers are shared, and growth must come from the heap (about 160 KB free).
- **Nothing large on the network task's stack.** A 4 KB JSON buffer declared inside a function overflowed the task's 12 KB stack, because the compiler inlined it into the handler that runs for every message. Big buffers now live in static memory.
- **Every message is classified on its raw bytes before parsing,** and each kind is parsed into a fixed-size buffer that keeps only the fields the panel uses. Memory use cannot grow with whatever Home Assistant decides to send.
- **Layout strings live in a fixed 2,560-byte arena,** so a layout can never take memory the interface needs. The current layout uses 275 bytes; a maximal one would use 2,320.

10.4 Touch coordinate handling

The panel is physically portrait (240 × 320) but the interface is landscape (320 × 240). The touch controller still reports coordinates in the physical orientation, so they are rotated in software in the touch driver. Without this, touches land mirrored or on the wrong axis — which looks like broken hardware but is purely a rotation mismatch.

10.5 Receiving and storing a layout

1. The designer fires a `homemesh_layout` event in Home Assistant; Home Assistant relays it to every subscriber, including the panel. A 700-byte layout arrives as a 955-byte message.
2. The panel recognises it from the raw message, before parsing, and parses it on its own path into a larger buffer, so the everyday path stays small.

3. It validates the whole layout **before storing anything**: limits, rooms, kinds, string sizes. A rejected layout is reported with its reason on the serial log, and the panel keeps running the layout it has.
4. An accepted layout is written to NVS and the board restarts into it. This is deliberate: rebuilding a whole screen in place fragments the 40 KB graphics pool, where running out freezes the screen; a restart takes a few seconds and always works.
5. At every boot the panel reads NVS. If nothing is stored, or it does not validate, it falls back to the layout compiled into the firmware, so a board never pushed to still comes up working.

Limits enforced by the firmware and the designer: **12 devices, 6 rooms, 6 tiles per page, 8 automations**. Pages do not scroll, so a seventh tile on a page would be unreachable.

10.6 The user interface

A header shows the page title, that room's temperature and humidity, and a connection dot (green: online; amber: Wi-Fi but no server; red: no Wi-Fi). Pages swipe sideways: a Home page of favourites, then one page per room, each with up to six tiles in two columns. Swiping up opens an Automations screen for scenes, scripts and automations.

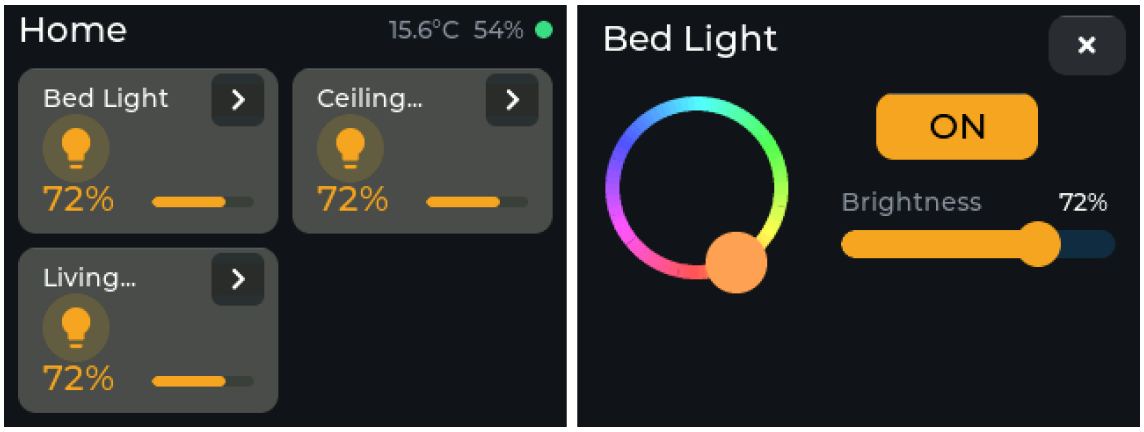


Figure 6. The Home page with lights on, and the detail panel for one light. Rendered by the panel's own interface code (Section 11.2), pixel for pixel.

- **A tile** shows the device name, its icon, its state, and for dimmable lights a brightness bar. A chevron opens the detail panel: on/off, a brightness slider and a colour wheel.
- **On state:** the tile lifts a shade, the icon turns amber in a tinted disc, and the reading turns amber. A dimmed light shows its level, such as 72%, rather than "ON". Six solid amber tiles on a page was noise; the lift is quieter and still unmistakable.
- **Long names** are cut to one line with "...". The designer shows the exact cut while you type.
- **Icons** are 16 Material Design glyphs compiled into fonts, so they cost flash, not graphics memory.
- **Fills are flat.** Gradients were tried and removed: the 16-bit display has only 32 levels of red and blue, so a subtle gradient showed as visible, tinted bands.

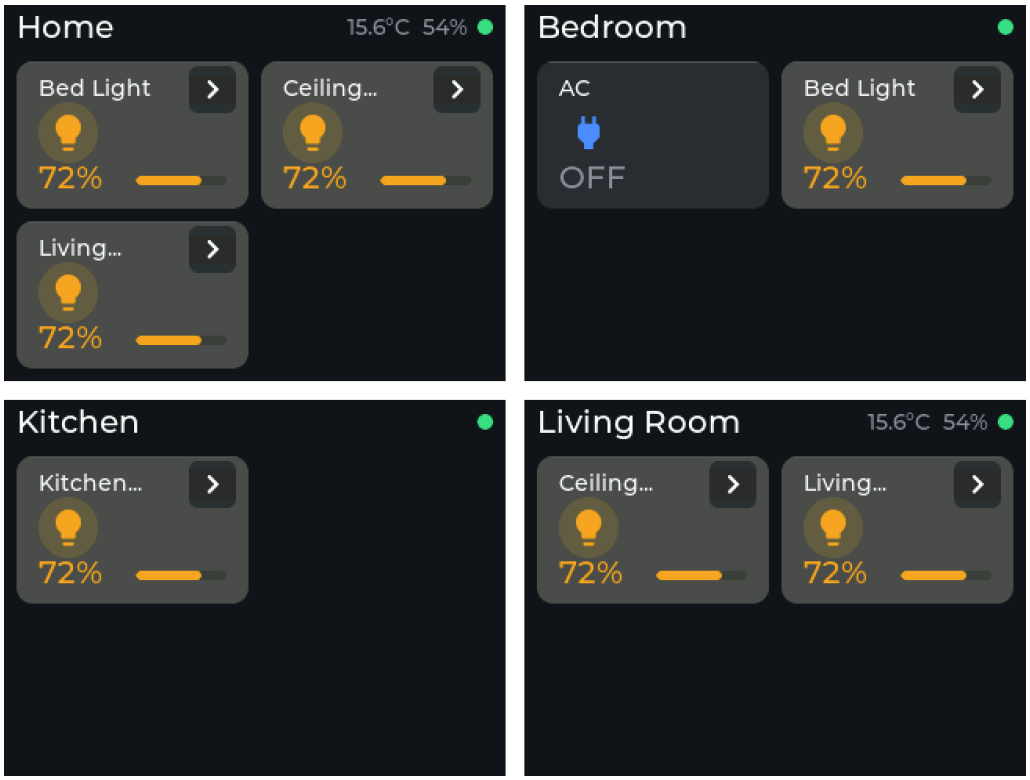


Figure 7. Every page of the current layout: Home and the three room pages. Rendered by the panel's own interface code.

10.7 Themes

Nine colours define the look: background, tile, on, accent, text, muted text, and the three link colours. A layout may carry any of them; a missing colour keeps its default, and the designer only sends colours that differ. Six presets are built in, including a light theme (Daylight) and a high-contrast one.

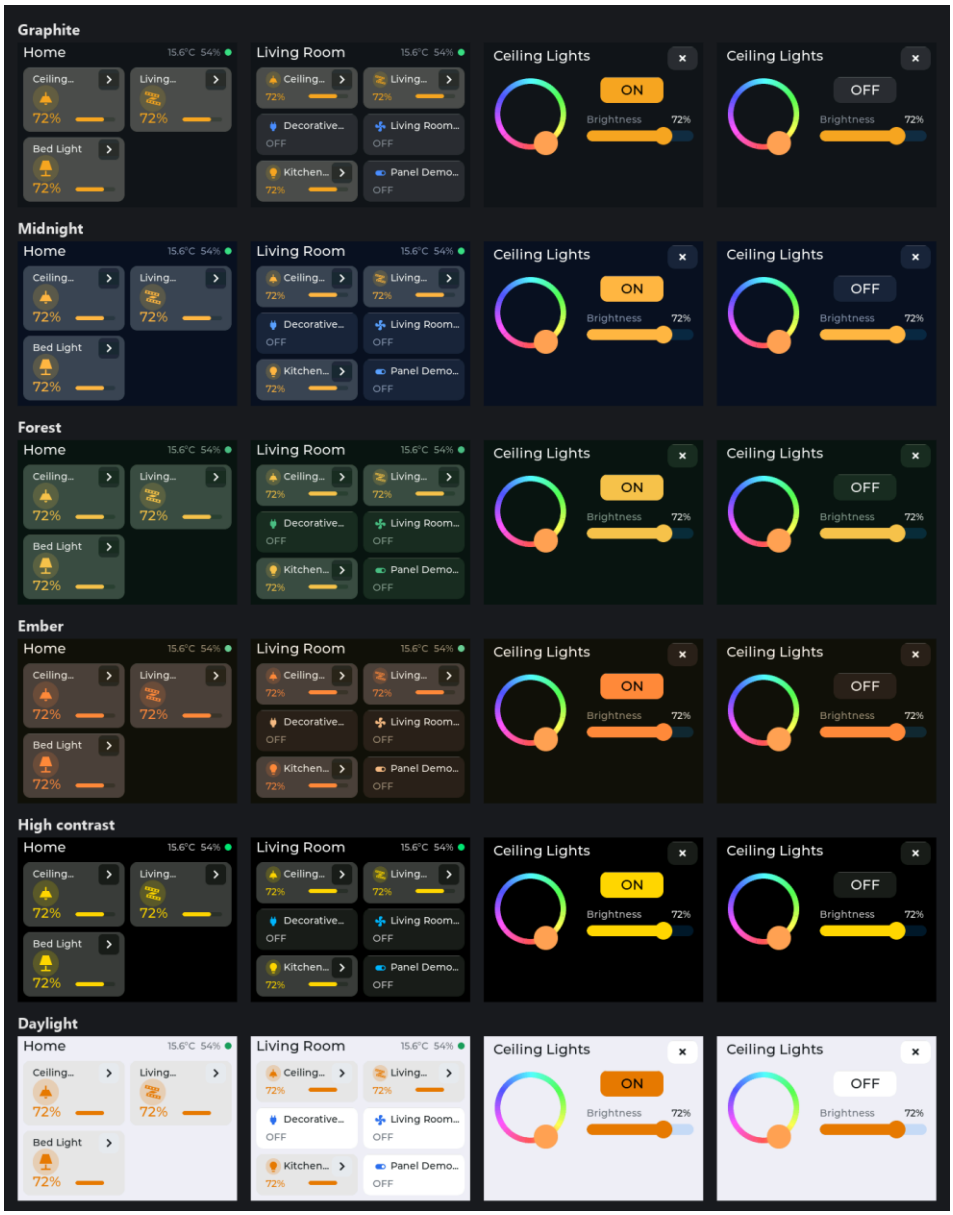


Figure 8. The six theme presets, each on the Home page, a room page with compact tiles, and the detail panel on and off. All rendered by the panel's own interface code.

10.8 A panel that explains its own failures

Two failures used to be silent. If the graphics library ran out of memory it froze the screen; it now prints the exact file and line with the memory state, then restarts. Wi-Fi failures are now logged with the driver's reason code — for example `reason 201 (NO_AP_FOUND)` means the panel cannot see the network at all. That one line is what cracked the network fault in Section 13.4.

11. The HomeMesh Designer (Desktop App)

The designer is the PC half of the split architecture: it discovers devices, lets the user design the panel, and sends the result to it. It replaced a simpler Python tool on 20 October.

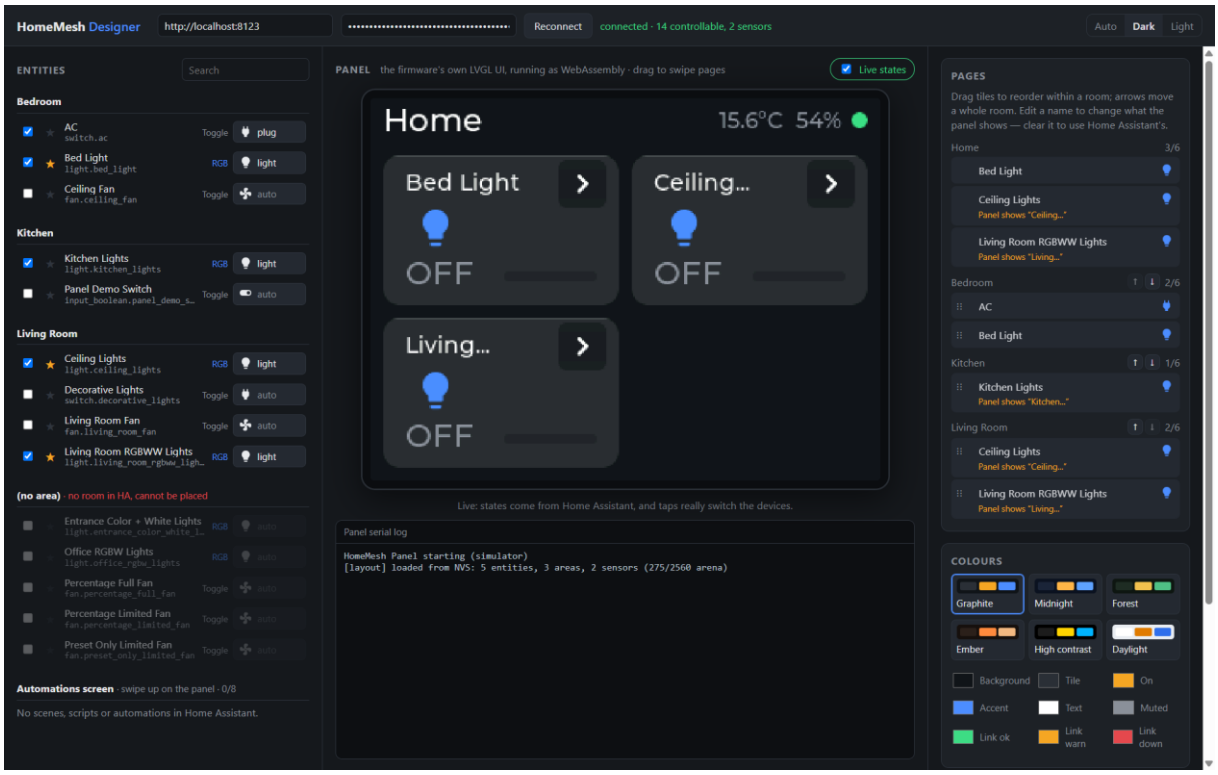


Figure 9. The designer connected to Home Assistant. Left: every entity, by room. Centre: the simulated panel and its log. Right: pages, names and colours.

11.1 What it does

- **Connects to Home Assistant** and lists every entity the panel can drive, grouped by room. Lights, switches, fans and helper switches are controllable; scenes, scripts and automations are actions; sensors are kept only if they measure temperature or humidity. Entities with no room are shown but cannot be placed, because the panel's pages are rooms.
- **Pick and arrange:** tick a device to place it, star it for the Home page, choose its icon, drag tiles to reorder within a room, and move whole rooms up and down.
- **Rename** any device for the panel. Each name shows exactly how the panel will cut it ("Ceiling...") and warns about characters the panel's font cannot draw, such as accents.
- **Colours:** six presets or any of the nine colours by hand.
- **Live states:** the simulated panel shows real device states, and tapping it really switches the devices, using the same commands the firmware sends.
- **Save to project** writes the layout file and the compiled fallback; **Push to panel** sends it through Home Assistant. Unsaved changes are flagged and can be reverted.
- Light, dark or automatic appearance for the app itself. Ships as a single portable .exe.

11.2 The simulator is the real interface

The preview is not a drawing of the panel. It is the panel's own interface code — `dashboard.cpp`, LVGL 8.3, the icon fonts and the layout validator — compiled to WebAssembly with Emscripten against the

firmware's own configuration. Same pixels, same fonts, same 16-bit colour, the same 40 KB graphics memory, and the same validator: if the simulator rejects a layout, the panel would too, with the same message. Nothing in the firmware source is copied or changed for it.

Applying a layout restarts the simulated panel, just as it restarts the real one. The name hints come from LVGL itself: an earlier approach that predicted the cut in JavaScript got three of eight test names wrong, because LVGL wraps at word boundaries, rounds each character to whole pixels and applies kerning.

11.3 How the designer is tested

- The layout format is written once, in Python (`ha_discover.py`). The app's version must produce **byte-identical** output from a stored Home Assistant snapshot, checked automatically.
- Name cuts are checked against cuts the firmware actually rendered.
- A live test drives a real light through Home Assistant and back.
- Every figure of the panel in this document was rendered by the same simulator.

12. System Operation

12.1 Network configuration

The demonstration runs on a Wi-Fi hotspot from the laptop, not on the college network, which breaks the system three ways: client isolation stops the panel reaching the laptop, a captive login page cannot be completed by a microcontroller, and local name lookup is filtered. The hotspot makes the demo independent of the venue.

Property	Value	Why it matters
Server address	192.168.137.1	Windows always gives the hotspot host this address, so it can be written into the firmware.
Frequency band	2.4 GHz only	The ESP32 has no 5 GHz radio. On this laptop the Wi-Fi card itself must be pinned to 2.4 GHz (Section 13.4).
Security	WPA2	WPA3 is unreliable on this chip with the Arduino framework.
Home Assistant	Docker on the laptop	Port 8123 on the hotspot address; Docker's own firewall rule covers it.

The hotspot is used for development too, not only for the demo. Had development used another network, the server address would change on the day, and because it is compiled into the firmware that would mean reflashing on the morning of the review.

12.2 Devices on the panel

The layout in the repository (`include/ha_layout.json`, read when this document was built) has 5 devices in 3 rooms, with 3 favourites on the Home page: Bed Light, Ceiling Lights, Living Room RGBWW Lights. The panel itself shows whatever was last pushed to it, and it prints which layout it loaded at boot.

Room page	Devices	Sensors
Bedroom	AC (switch, on/off) Bed Light (light, brightness and colour)	none
Kitchen	Kitchen Lights (light, brightness and colour)	none
Living Room	Ceiling Lights (light, brightness and colour) Living Room RGBWW Lights (light, brightness and colour)	Temperature, Humidity

Rubric minimum, all met: at least three lights with at least one brightness slider (every light has one), a switch, a temperature and humidity readout, and a connection indicator.

12.3 Changing what the panel shows

1. Open the designer. It connects to Home Assistant and lists the devices.
2. Tick, star, rename, reorder and colour. The simulated panel updates as you go.
3. Press **Push to panel**. The panel restarts and comes back with the new layout in a few seconds.
4. Optionally **Save to project**, so the compiled fallback matches.

13. Testing and Results

The system was tested end to end against a real Home Assistant installation (version 2026.9, in Docker on a Windows 11 laptop) whose demonstration integration provides lights, switches, fans and sensors — 123 entities in all. Server-side figures were measured with a script on the laptop; panel figures come from the panel's serial log and the build report. Measured 18 October.

13.1 Message size and latency

Measurement	Result
Full state dump, get_states (123 entities)	57,671 bytes
Average size per entity	469 bytes
Panel's initial state, subscribe_entities (7 entities)	2,403 bytes, one message
Change message: brightness only / light off	144 bytes / 262 bytes
Layout delivery (700-byte layout)	955-byte message
Snapshot delivered after subscribing (server)	11.5 ms
Command to state-change message, median of 10 (server)	3.2 ms
Panel round-trip time over the hotspot, mean of 6	40 ms, 0% loss

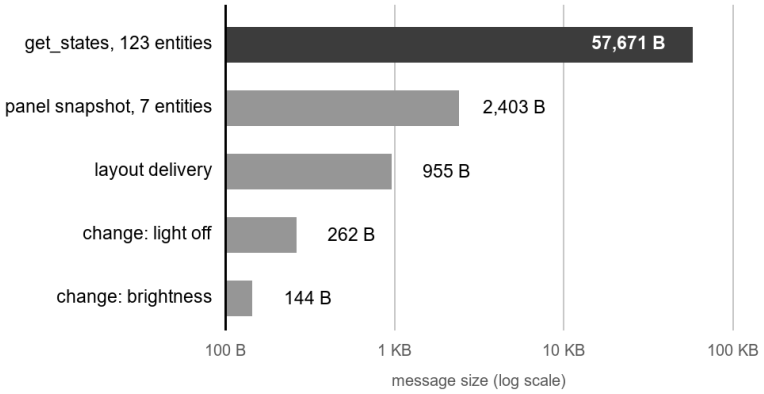


Figure 10. Message sizes: what the panel avoids, and what it handles.

The initial state is 24 times smaller than the full dump and, more importantly, depends on the layout rather than the size of the house. A command produces its state change in milliseconds — far below the point where a touch response feels delayed.

13.2 Resource usage

Resource	Used / available
Flash (application partition)	1,198,037 / 3,145,728 bytes (38.1%)
Static RAM	123,024 bytes; about 1.5 KB below the linker limit
Graphics pool, free after the first full draw	15,652 / 40,960 bytes
Free heap after start-up (lowest)	161,028 bytes (158,632)

Resource	Used / available
Network task stack, free after login	8,272 / 12,288 bytes
Icon fonts (16 glyphs, two sizes)	about 7.6 KB of flash, 0 bytes of graphics memory

13.3 Functional verification

Test	Result
Connect and log in to Home Assistant over the hotspot	Pass
Receive the initial state of every device in one message	Pass
Switch a device from the panel; it changes in Home Assistant	Pass
Change a device in Home Assistant; the panel follows	Pass
Set brightness and colour from the detail panel	Pass
Show temperature and humidity on the right room page	Pass
Swipe pages during network traffic	Pass — no stutter
Stop the server; the panel shows it	Pass — tiles greyed out, no stale values
Push a layout from the designer; the panel stores it and restarts into it	Pass — on the board, 17 October
Reject an invalid layout and keep the old one	Pass — reason printed on the log
Boot with nothing stored; fall back to the compiled layout	Pass — on the board, 16 October
Designer: tap the simulated panel in Live mode; the real light switches	Pass — 20 October
Designer layout output identical to the reference tool	Pass — automated test

13.4 Field case study: the connection that kept dropping

For days the panel would randomly show no connection, tiles read "--", and faded tiles seemed to linger between pages. It looked like flaky firmware. It was two problems stacked on each other.

1. **A design flaw that amplified faults.** The panel fetched each device's starting state with an HTTP request made from inside the WebSocket's own message handler. When those requests were slow — up to 21 seconds for seven devices — the connection's heartbeat could not run, so the library dropped the connection, reconnected and did it all again. **Fix:** take the starting state from the subscription on the existing connection. Nothing can block the handler any more, and the HTTP library and 34 KB of flash went with it.
2. **The root cause was the laptop.** The panel then failed to join the network at all, and the new logging said why: `reason 201 (NO_AP_FOUND)`. The laptop's Wi-Fi card has a single radio, so the hotspot must share the channel of the laptop's own Wi-Fi connection. Whenever the router moved the laptop to 5 GHz, the hotspot silently moved with it — whatever its own setting said — and became invisible to the 2.4 GHz-only panel. The router changes band as it pleases, which is why it looked random. **Fix:** pin the card to 2.4 GHz in Device Manager. The panel joined at −35 dBm with no packet loss, and the preflight script now checks the setting.

The lesson worth presenting: a flaky network alone would have cost a few readings; blocking inside the message handler turned each slow reply into a dropped connection. And logging that names its causes turned a days-long mystery into a one-session diagnosis.

13.5 Other challenges and their resolution

Challenge	Resolution
Board carried different display and touch controllers than documented	Identified both by testing; confined hardware settings to two driver files.
Panel rebooted every few seconds under network load	Network task starved the idle task and tripped the watchdog. Priority lowered to 0.
Touches landed in the wrong place	Touch reports physical orientation; rotation added in the driver.
Program too big for the application partition	Switched to a 3 MB partition, dropping the unused over-the-air slot.
Crash at start-up inside an unrelated print statement	A 4 KB buffer inlined onto the network task's stack. Moved to static memory.
Layout pushes vanished with no error	A fixed-size JSON filter silently dropped a key that did not fit. Pushes are now recognised from the raw message, before parsing.
Larger graphics pool would not link	Static memory, not total RAM, is the limit. Buffers shared; growth must come from the heap.
Gradients showed visible bands	16-bit colour has 32 levels of red and blue. Fills made flat.
Long names wrapped into the icon	Cut to one line with "..."; the designer shows the cut.
Light theme: names vanished on lit tiles, invisible close button	LVGL's default theme forces white text on checked buttons; colours now set explicitly. Found by rendering every preset in the simulator.
Panel joined Wi-Fi but could not reach the server	The firmware had the house network's name, not the hotspot's. Corrected.
Most Home Assistant devices invisible	The reason for leaving MQTT (Section 5.1).

14. Future Scope

- **Encrypted connection and token provisioning.** The connection is unencrypted and the access token is compiled in — acceptable on an isolated hotspot, not in a real home.
- **Multi-cell widgets:** a two-wide tile with its controls inline instead of behind the chevron. Needs real placement in the layout format, the firmware and the designer.
- **Acknowledged layout delivery,** so the designer can confirm the panel took a layout. Today only the panel's serial log confirms it.
- **Use the microSD card** for logging or images, once it has been tested.
- **A dedicated ESP32 access point** so the panel does not depend on a laptop hotspot. Written, not yet run.
- **Remove the legacy MQTT code** to shrink what can break.
- More device types — media players, covers, thermostats — multiple panels with their own layouts, and a screen sleep mode.
- End-to-end latency measured from touch to a physical device.

15. Conclusion

The project set out to build a wall-mounted touchscreen that controls a home automation system entirely over the local network, on hardware costing a small fraction of a commercial panel. That has been met and demonstrated against a real installation: the panel switches, dims and colours lights, shows

temperature and humidity, follows changes made anywhere else in the house, and stays responsive while the network is busy, with no cloud service involved.

Three decisions account for most of its value. **Replacing MQTT with the Home Assistant WebSocket API** changed device coverage from a subset that depended on how each device was set up to every device in the system. **Recognising that on-device discovery was impossible** and splitting the work — the PC discovers, the panel subscribes — keeps every message small: 2,403 bytes where the naive design needed 57,671 for a small house. And **making the layout data** means the panel is redesigned in a desktop app and updated over the network in seconds, with a preview that is the panel's own code.

The field case study adds a fourth lesson: firmware that reports why it fails is worth more than firmware that tries to hide it.

16. References

16.1 Published literature

Each entry was checked against the Crossref DOI registry. The same list, with in-text citations, is in the IEEE paper.

[1] M. J. C. S. Reis and C. Serôdio, "Edge AI for real-time anomaly detection in smart homes," *Future Internet*, vol. 17, no. 4, Art. no. 179, 2025, doi: 10.3390/fi17040179.

[2] S. N. Swamy, P. Shivanarayanamurthy, G. Purohit, and K. S. Raju, "An edge computing-assisted Internet of Things-based secure smart home," in *Lecture Notes on Data Engineering and Communications Technologies*. Singapore: Springer, 2022, pp. 159–173, doi: 10.1007/978-981-16-7182-1_14.

[3] D. Russo, V. Miori, G. Tolomei, and D. Belli, "Home automation interoperability: Two decades of lessons learned and future prospects into the development of IoT ecosystems," *Internet of Things*, vol. 37, Art. no. 101906, 2026, doi: 10.1016/j.iot.2026.101906.

[4] S. Madadi-Barough, P. Ruiz-Blanco, J. Lin, R. Vidal, and C. Gomez, "Matter: IoT interoperability for smart homes," *IEEE Commun. Mag.*, vol. 63, no. 4, pp. 106–112, 2025, doi: 10.1109/MCOM.001.2400274.

[5] C. Stolojescu-Crisan, C. Crisan, and B.-P. Butunoi, "An IoT-based smart home automation system," *Sensors*, vol. 21, no. 11, Art. no. 3784, 2021, doi: 10.3390/s21113784.

[6] G. Verma, S. Pachauri, A. Kumar, D. Patel, A. Kumar, and A. Pandey, "Smart home automation with smart security system over the cloud," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, 2023, pp. 1–7, doi: 10.1109/ICCCNT56998.2023.10306548.

[7] R. Tse, B. L. Muhammad-Bello, and I. A. Salihu, "Design and prototype implementation of an ESP32-based web smart home automation system for remote appliance control," *UMYU Scientifica*, vol. 5, no. 3, pp. 44–58, 2026, doi: 10.56919/usci.2653.004.

[8] C. Veigl, B. Klaus, B. Aigner, and M. Wagner, "Universal access panel: A novel approach for accessible smart homes and IoT," in *Lecture Notes in Computer Science*. Cham: Springer, 2022, pp. 148–158, doi: 10.1007/978-3-031-08645-8_18.

[9] P. Gupta and I. O. Prabha M, "A survey of application layer protocols for Internet of Things," in *Proc. Int. Conf. Commun. Inf. Comput. Technol. (ICCICT)*, 2021, pp. 1–6, doi: 10.1109/ICCICT50803.2021.9510140.

[10] P. K. Donta, S. N. Srirama, T. Amgoth, and C. S. R. Annavarapu, "Survey on recent advances in IoT application layer protocols and machine learning scope for research directions," *Digit. Commun. Netw.*, vol. 8, no. 5, pp. 727–744, 2022, doi: 10.1016/j.dcan.2021.10.004.

[11] C. Bayılmış, M. A. Ebleme, Ü. Çavuşoğlu, K. Küçük, and A. Sevin, "A survey on communication protocols and performance evaluations for Internet of Things," *Digit. Commun. Netw.*, vol. 8, no. 6, pp. 1094–1104, 2022, doi: 10.1016/j.dcan.2022.03.013.

[12] Lalhriatpuii, Ruchi, and V. Wasson, "Comprehensive exploration of IoT communication protocol: CoAP, MQTT, HTTP, LoRaWAN and AMQP," in *Communications in Computer and Information Science*. Cham: Springer, 2024, pp. 261–274, doi: 10.1007/978-3-031-75861-4_23.

[13] M. Mishra and S. R. N. Reddy, "Practical assessment of application layer protocols for resource constraint IoT applications," in *Smart Innovation, Systems and Technologies*. Singapore: Springer, 2021, pp. 549–559, doi: 10.1007/978-981-16-0878-0_54.

[14] S. Kaur and V. Khanna, "Implementation and comparison of MQTT, WebSocket, and HTTP protocols for smart room IoT application in Node-RED," in *Internet of Things*. Cham: Springer, 2022, pp. 165–193, doi: 10.1007/978-3-030-89554-9_8.

[15] O. Ajayi, A. Bagula, J. Bode, and M. Damon, "A comparison of publish-subscribe and client-server models for streaming IoT telemetry data," in *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*. Cham: Springer, 2023, pp. 129–139, doi: 10.1007/978-3-031-35883-8_9.

[16] M. I. Yamin, A. Ariman, R. M. Dinata, and N. Purnomo, "Comparative performance evaluation of MQTT, WebSocket, and Firebase on ESP32-C3 for real-time IoT communication," *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 10, no. 3, pp. 3917–3923, 2026, doi: 10.36040/jati.v10i3.18056.

[17] H. Demir, I. Selcuk, and A. E. Cosgun, "Power consumption analysis of ESP32 and Arduino Nano for low-power applications," in *Proc. 9th Int. Symp. Innov. Approaches Smart Technol. (ISAS)*, 2025, pp. 1–4, doi: 10.1109/ISAS66241.2025.11101965.

16.2 Technical documentation

[18] Home Assistant. WebSocket API. developers.home-assistant.io/docs/api/websocket

[19] Home Assistant. Architecture: entities and the entity registry. developers.home-assistant.io/docs/architecture_index

[20] Espressif Systems. ESP32-WROOM-32 Datasheet.

[21] Espressif Systems. ESP-IDF FreeRTOS reference: task priorities and the task watchdog.

[22] LVGL. Light and Versatile Graphics Library, version 8.3 documentation. docs.lvgl.io/8.3

[23] Bodmer. TFT_eSPI: TFT graphics library for ESP8266 and ESP32. github.com/Bodmer/TFT_eSPI

[24] Links2004. arduinoWebSockets: WebSocket server and client for Arduino. github.com/Links2004/arduinoWebSockets

[25] B. Blanchon. ArduinoJson: JSON library for embedded C++. arduinojson.org

[26] Sitronix Technology. ST7789V TFT controller/driver datasheet.

[27] Goodix Technology. GT911 capacitive touch controller datasheet.

[28] OASIS. MQTT version 3.1.1 specification.

[29] PlatformIO. Espressif 32 platform documentation. docs.platformio.org

[30] Emscripten. Compiling C and C++ to WebAssembly. emscripten.org/docs

[31] Electron. Electron documentation. electronjs.org/docs

Appendix A — Suggested Presentation Plan

Twenty slides. The content for each is drawn from the section shown, so nothing needs writing from scratch. Figure files are in `docs/figures/` and `docs/paper/`.

#	Slide	What to put on it	Source / figure
1	Title	Project title, course code, team members with register numbers, guide, date.	Title page
2	Introduction	Phone apps are slower than the switch they replace; a panel is always on and always in one place. No cloud.	§2
3	Literature survey	The table, shortened to one line per row.	§2.2
4	Problem statement	The six conditions, one short line each.	§3
5	Objectives	The eight objectives, one line each.	§4
6	Existing systems	The three approaches and their limitations.	§5
7	Why not MQTT	**Key slide.** Same Zigbee bulb: reachable or invisible depending only on how it was added. Then the three designs table.	§5.1, §5.2
8	Proposed system	WebSocket API: every device, one command.	§6
9	The memory wall	**Key slide.** 57,671 bytes vs a 40 KB pool. PC discovers, panel subscribes. 2,403 bytes.	§6.1, paper_message_sizes.png
10	Overall architecture	Full width. Setup path first, then runtime.	arch_overall.png
11	Hardware	Diagram plus the components table. Mention the controllers had to be identified by testing.	arch_hardware.png, §8
12	Software	The two cores and the queue between them.	arch_software.png
13	Protocol	The five messages. State in, commands out, layouts in.	§10.1, paper_sequence.png
14	Layout as data	Designer → Home Assistant event → validate → NVS → restart. No reflash.	§6.2, §10.5
15	The interface	Tiles, on state, detail panel; the themes.	ui_tile_detail.png, theme_presets.png
16	The designer	Screenshot. The preview is the panel's own code in WebAssembly; Live mode switches real devices.	designer_app.png, §11
17	Results	Message size table and resource table.	§13.1, §13.2
18	Field case study	**Best story.** Random disconnects → blocking handler + single-radio Wi-Fi on 5 GHz. Found because the panel logs why it fails.	§13.4
19	Future scope	Encryption, multi-cell tiles, acknowledged push, microSD, dedicated access point.	§14
20	Conclusion + demo	The three decisions. Then the live demonstration.	§15

Figure files

Figure	File
Overall architecture	docs/figures/arch_overall.png
Hardware architecture	docs/figures/arch_hardware.png

Figure	File
Software architecture	docs/figures/arch_software.png
Block diagram	docs/figures/img/paper_block_diagram.png
Message sequence	docs/figures/img/paper_sequence.png
Message sizes chart	docs/figures/img/paper_message_sizes.png
Panel: Home page and detail panel	docs/figures/img/ui_tile_detail.png
Panel: every page	docs/paper/fig_ui_pages_lvgl.png
Panel: six themes	docs/figures/img/theme_presets.png
Designer app	docs/figures/img/designer_app.png

Presentation notes

- Slides 7, 9 and 18 carry the engineering argument. If time is short, cut elsewhere.
- Use the architecture figures full width; shrunk, their labels become unreadable.
- The strongest live moment: change a light in Home Assistant's web page and let the audience watch the panel follow — two-way sync in a way no slide can show. The second strongest: rename a device in the designer, press Push, and watch the panel restart with the new name.
- Panel images in this document are rendered by the panel's own code, so they are exact — but photograph the real panel for at least one slide, so nobody thinks it is a mock-up.

Anticipated questions

Question	Answer
Why not a Raspberry Pi with a browser?	Cost, boot time and power. This starts in a couple of seconds and draws a fraction of the power, with no operating system to maintain on a wall.
What happens if Home Assistant goes down?	The connection drops, the panel sees it immediately and greys the tiles out. It never shows stale values that look live.
Are these real devices?	They come from Home Assistant's demonstration integration. The protocol is identical to a physical bulb — the panel cannot tell the difference, and real hardware needs no firmware change.
How do you change what is on the panel?	In the designer: tick, star, rename, reorder, pick colours, Push. The panel restarts with the new layout in seconds. No cable, no reflash.
Why does it restart when a layout arrives?	Rebuilding a screen in place fragments the 40 KB graphics memory, and running out freezes the screen. A restart takes seconds and always works.
Is the designer's preview a mock-up?	No — it is the panel's own interface code compiled to WebAssembly. Same fonts, same pixels, same memory limit, same validator.
What was the hardest bug?	The random disconnects in §13.4: a blocking request in the message handler, plus the laptop's single-radio Wi-Fi card dragging the hotspot onto 5 GHz.
Why LVGL 8.3 and not 9?	Version 9 changed the driver interfaces, and nearly every example for this board is written against 8.3.
How did you know which display controller it has?	By testing. This board ships with different controllers in different batches.
How many devices can it handle?	12 devices, 6 rooms, 6 per page, 8 automations. Pages do not scroll, so a seventh tile would be unreachable.

Question	Answer
Is it secure?	On an isolated hotspot, yes enough. The connection is unencrypted and the token is in the firmware, so a real home would need TLS — listed as future work.
What is the microSD card for?	Nothing yet; it has not been tested. Logging is the likely use.

Appendix B — Internal Note (Not for Presentation)

This appendix is for the team only

Do not show this page, read it aloud, or put it on a slide. It records what is not finished, so that nothing is claimed during the review that cannot be demonstrated.

What is genuinely working

All of these can be demonstrated live and claimed without qualification:

- The panel controls a real Home Assistant installation over the local network, with no cloud.
- Two-way sync: a change in Home Assistant appears on the panel, and the panel's taps change Home Assistant.
- Brightness and colour control on every light; temperature and humidity; the connection indicator.
- Smooth page swiping during network traffic.
- Layouts pushed over the network and stored on the panel, with no reflash — proven on the board on 17 October.
- The designer: discovery, picking, favourites, icons, reordering, renaming, themes, the simulated panel, and Live mode switching real devices.

What must not be claimed

Do not say	Say instead
That MQTT has been removed	The MQTT code is still in the project, unused, kept as a fallback.
That the microSD card is used or works	It is fitted but has not been tested.
That the connection is encrypted	It is not; that is future work (§14).
That the designer knows the panel received a layout	There is no acknowledgement. "Pushed" means Home Assistant accepted it; the panel's serial log is the only confirmation.
That latency was measured from touch to bulb	Latency was measured at the server (3.2 ms) and as network round trip (40 ms).
That the ESP32 access point works	It is written but has never been run.
That tiles can be resized or placed freely	Tiles are placed by order in a fixed two-column grid.

Check before the review: firmware on the board

The board is running the 18 October firmware

Fixes made on 20 October are built and tested in the simulator but **not yet flashed**: long names wrapping into the icon, and three bugs that only show on the light (Daylight) theme.

Either flash `../homemesh_ha_firmware_20oct.bin` before the review and check the boot log, or demo with a dark theme and short names.

The new designer's Push sends the same event as the old tool (tested byte for byte), but nobody has yet watched a push from *the new designer* arrive on the board. Do it once with the serial monitor open and look for `[layout] push received` then `[layout] loaded from NVS`.

The Automations screen needs at least one scene, script or automation in Home Assistant. The demo installation has none, so do not promise it unless one is added.

Two quirks that may be noticed, and how to answer

If someone notices	The answer
Every light is listed as colour-capable (RGB), not dimmable	Correct, not a fault. RGB means on/off plus brightness plus colour, so every light gets a slider. The requirement was at least one.
Bedroom and Kitchen show no temperature or humidity	The demo installation has one temperature and humidity pair, in the Living Room. Showing nothing beats showing an invented reading.

Before the presentation

```
powershell -ExecutionPolicy Bypass -File tools\preflight.ps1
```

- Run it from cold. It checks the Wi-Fi card is pinned to 2.4 GHz, the hotspot is up, the firewall path, that Home Assistant is running and reachable on 192.168.137.1, the token, and every device in the layout. It names the fix for anything that fails.
- If the panel does not connect, read its serial log first. `reason 201 (NO_AP_FOUND)` means it cannot see the network — almost certainly the hotspot is on 5 GHz: check Band Selection and toggle the hotspot.
- If the venue Wi-Fi is unusable, the hotspot still needs a network to share: join the laptop to a phone hotspot set to 2.4 GHz first.
- If something is badly wrong: `git reset --hard v0.4-firmware-freeze`, or flash `../homemesh_ha_firmware_18oct_full.bin` (restores the whole board; push the layout again afterwards), or restore from GitHub.